

FULL STACK INTERN TASK SUBMISSION

Resume Shortlister

A full-stack resume screening and candidate ranking web application that compares uploaded resumes with a job description, generates a match score, and displays ranked candidates for faster HR screening.

CANDIDATE Kamal Bhagat	PROJECT TYPE Full-stack web application
GITHUB REPOSITORY github.com/ka-sr/Resume-shortlister	LIVE APPLICATION URL Add Render deployment URL here

1. Project Objective

The objective of this project is to simplify the initial HR screening process. Recruiters can upload one or more resumes, provide a job description, and get a ranked candidate list based on suitability for the role.

Upload Resumes	Add Job Description	Extract Text	Score Candidates	Show Ranking
---------------------------	--------------------------------	---------------------	-----------------------------	---------------------

2. Key Features

- Multiple resume upload support for PDF, DOC, and DOCX files.
- Job description can be entered manually or uploaded as a document.
- Resume and JD text extraction using backend parsers.
- Candidate match score from 0 to 100.
- Ranking from highest score to lowest score.
- Candidate name, contact details, resume preview, score, rank, matching skills, and missing skills.
- Search and sort candidates from the dashboard.
- CSV export for spreadsheet review.
- Responsive UI with upload progress and score badges.
- Automated tests for scoring, ranking, name extraction, and contact extraction.

3. Tech Stack

Layer	Technology
Frontend	React, Vite, CSS, Lucide icons
Backend	Node.js, Express.js
File Upload	Multer
Parsing	pdf-parse for PDF, Mammoth for DOCX, best-effort text fallback for DOC
Storage	Local JSON by default, optional PostgreSQL through DATABASE_URL
Testing	Node.js built-in test runner
Deployment	Render-ready using render.yaml

4. Architecture Overview

The application follows a simple client-server architecture. The React frontend handles the upload workflow, dashboard, sorting, searching, previews, and CSV export. The Express backend receives uploaded files, extracts text, applies scoring logic, ranks candidates, and stores the analysis run.

File / Folder	Purpose
<code>src/App.jsx</code>	Main React dashboard and upload workflow.
<code>src/styles.css</code>	Responsive UI styling, cards, score badges, upload progress, and preview layout.
<code>server/index.js</code>	Express API, upload handling, analysis endpoint, static frontend serving.
<code>server/textExtraction.js</code>	Extracts text from PDF, DOCX, DOC, and TXT files.
<code>server/scoring.js</code>	Skill extraction, keyword extraction, contact extraction, and score calculation.
<code>server/ranking.js</code>	Sorts candidates by score and assigns rank.
<code>server/db/store.js</code>	Stores analysis history locally or in PostgreSQL.

tests/scoring.test.js	Automated tests for scoring and ranking behavior.
-----------------------	---

5. Candidate Scoring Approach

The project uses a deterministic scoring approach so the result is explainable and easy to review. The scoring model compares each resume against the job description across multiple factors.

Factor	Weight	Description
Skills Match	45 points	Checks how many required JD skills are present in the resume.
Keyword Similarity	25 points	Compares important keywords from the JD with resume keywords.
Experience Relevance	15 points	Compares required years of experience with detected resume experience.
Education Alignment	10 points	Checks education terms such as degree, bachelor, engineering, masters, etc.
Role Fit	5 points	Checks title and role-specific keywords for additional relevance.

Final score is capped between 0 and 100. Candidates are ranked by score in descending order. If two candidates have the same score, names are used as a tie-breaker for stable ordering.

6. Testing

The project includes automated tests to improve reliability and make the submission more professional. The test command is:

```
npm test
```

- Skill extraction test.
- Relevant resume scoring test.
- Weak resume scoring test.
- Candidate ranking order test.
- Candidate name and contact extraction test.

7. Setup Instructions

Install dependencies and run the development app:

```
npm install
```

```
npm run dev
```

Frontend: `http://127.0.0.1:5173`

API: `http://127.0.0.1:4000`

Production Run

```
npm run build
```

```
npm start
```

8. Deployment

The project is ready for Render deployment. A `render.yaml` file is included with the build command, start command, free plan, and health check path.

- Build command: `npm ci && npm run build`
- Start command: `npm start`
- Health check: `/api/health`

Note: Please replace "Add Render deployment URL here" with the final live URL after Render deployment is complete.

9. Assumptions and Limitations

- Candidate name is detected from the first clean person-like line, otherwise filename is used.
- DOCX and PDF parsing are more accurate than legacy DOC parsing.
- CSV export is used because Excel can open CSV files directly.
- Uploaded files are stored on the server filesystem. For production, persistent object storage is recommended.
- The current scoring is rule-based and explainable. It can be upgraded later with AI/LLM scoring if needed.

10. Future Improvements

- Add authentication for recruiter/admin access.
- Add persistent cloud file storage such as S3 or Cloudinary.
- Add Excel XLSX export.

- Add AI-based semantic scoring as an optional mode.
- Add detailed history page for previous screening runs.

Prepared for Full Stack Intern Task Submission - Resume Shortlister.